Shape Analytics

Physics, numerics and code

A complete technical report on the solid-mechanics analysis app: the principles of the Mechanics of Solids course (Chapters 1–11), how each is implemented in the source files, and how the results are verified

Project	`shape-analytics` (FastAPI, Three.js, Plotly, numpy)
Scope	3D stress field, section analytics, nine analytical simulators, finite element tab
Companion files	`REPORT.md`, `docs/physics_report.pdf`, `docs/fem_primer.pdf`

Code excerpts in this report are taken directly from the source files; the numbers beside them are the line numbers in those files at the time of writing. Greek letters and mathematical symbols in the source (for example `sigma` for the Greek letter) are written out in ASCII in the listings so that they print.

Contents

1 Introduction and architecture **2**

1.1 What the project is . . . 2
1.2 Conventions . . . 2
1.3 File map . . . 2
1.4 How the pieces fit . . . 3
1.5 Running the project . . . 3

2 Stress at a point **4**

2.1 The stress tensor and Cauchy's formula (syllabus Chapters 1–2) . . . 4
2.2 Principal stresses and invariants (Chapter 4) . . . 4
2.3 Equivalent stresses . . . 5
2.4 Mohr's circle simulator (Chapters 2 and 4) . . . 6

3 Half-space elasticity: the equations behind the 3D field **7**

3.1 Governing equations (Chapters 3, 5, 6) . . . 7
3.2 Normal point load: Boussinesq . . . 7
3.3 Tangential point load: Cerruti . . . 7
3.4 Equivalent depth: removing the singularity . . . 8
3.5 Superposition (Chapter 10.1) . . . 8

4 The 3D stress field pipeline **9**

4.1 From geometry to loads . . . 9
4.2 A local frame for each load . . . 9
4.3 Superposition and rotation to global axes . . . 10
4.4 Full-surface pressure . . . 11
4.5 Quantities displayed . . . 11

5 Section analytics **13**

5.1 What a section shows . . . 13
5.2 Server-side evaluation . . . 13
5.3 Front end . . . 13

6 The analytical simulators **15**

6.1 The framework in `sims.js` . . . 15
6.2 Beams (Chapters 8 and 9) . . . 15
6.3 Shaft torsion (Chapter 7) . . . 16
6.4 Axial loading (Chapters 6 and 7) . . . 16
6.5 Pressure vessels (Chapter 7) . . . 17
6.6 Thermal stress (Chapter 6.6) . . . 17
6.7 Failure theories (Chapter 11) . . . 18
6.8 Energy methods (Chapter 10) . . . 18

6.9 Column buckling (Chapter 9.4) . . . 18

7 The finite element method **20**

7.1 Why a finite element tab . . . 20
7.2 The method in seven steps . . . 20
7.3 One-dimensional elements . . . 20
7.4 Two-dimensional elements . . . 21
7.5 The three-dimensional block . . . 22
7.6 The FEM user interface . . . 23

8 Verification **25**

8.1 Approach . . . 25
8.2 Limiting cases of the simulators . . . 26
8.3 Verification of the finite element code . . . 26
8.4 Defects found by the tests, and what was changed . . . 26

9 Limitations and how to extend the project **28**

9.1 Limits of validity . . . 28
9.2 Adding a simulator . . . 28
9.3 Removing the FEM tab . . . 28
9.4 Ideas for further work . . . 28

A Index: physics principle to code **29**

B Interfaces and data formats **30**

B.1 HTTP API . . . 30
B.2 Symbols . . . 30

1 Introduction and architecture

1.1 What the project is

Shape Analytics is a browser application, served by a small Python server, that lets a student load a solid body and see the resulting stress, strain and deformation. It contains four groups of tools:

1. a **3D stress field**: a cuboid, cylinder or sphere loaded by a pressure or traction painted on its surface, shown as coloured stress maps on a deformed mesh (Chapters 2–6 of the course);
2. **section analytics**: a cut through the same body that shows the stress inside it;
3. **nine analytical simulators**: Mohr's circle, beams, shaft torsion, axial loading, pressure vessels, thermal stress, failure theories, energy methods and column buckling (Chapters 2–11);
4. a **finite element tab**: a numerical solver for a bar, a beam, two plane-stress problems and a 3D block (not in the syllabus, but built from its equations).

The purpose of this report is to show, for each tool, *which physical principle is being used, where in the source code it lives, and how the code turns the principle into numbers.* Each chapter therefore has the same shape: the physics first (with the equations from the course), then the code (short excerpts quoted from the files, with their line numbers), then notes on limits and checks.

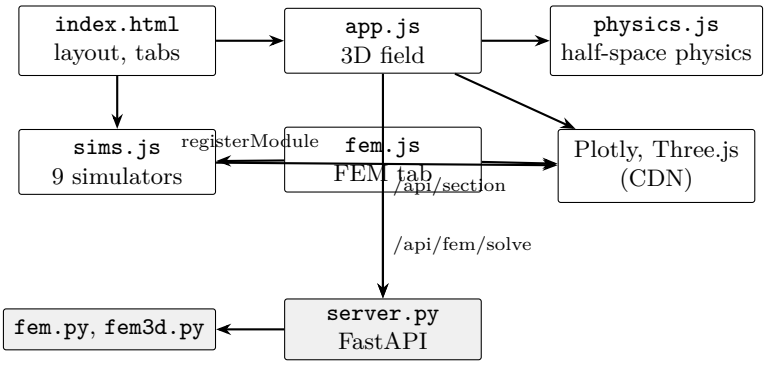
1.2 Conventions

Sign convention	tension is positive; compressive loads give negative normal stress. In the half-space frames z points *into* the body.
Units, 3D field	lengths in metres, forces in newtons, stresses in pascals inside the physics functions, shown as MPa in the interface.
Units, simulators	millimetres, newtons, megapascals (so E is in MPa); loads shown in kN, moments in kN m.
Units, FEM	SI inside the solver; the interface uses mm, kN and MPa.
Tensor storage	$[\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \tau_{xy}, \tau_{xz}, \tau_{yz}]$ in `physics.js`; the FEM 3D code orders $[xx, yy, zz, yz, xz, xy]$ (engineering shear strain).
Small strain	every model is linear elastic and assumes $\lvert\nabla \mathbf{u}\rvert \ll 1$.

1.3 File map

File	Lines	Role
`index.html`	238	page structure: header, module navigation, the 3D viewport, the analytics drawer, the simulator layout
`style.css`, `fem.css`	437, 19	the design tokens, light and dark themes, layouts
`app.js`	927	the 3D field: geometry, painting, loads, the stress computation, the section plots, theme
`physics.js`	98	pure functions for half-space stresses, displacements, principal stresses, von Mises, strain
`sims.js`	804	the nine simulators as configuration objects, plus the framework that renders them
`fem.js`	354	the FEM tab: controls, request, mesh drawing, charts, explanation panels
`server.py`	365	FastAPI app: `/api/traction`, `/api/principal`, `/api/section`, static files
`fem.py`	508	finite element solvers (bar, beam, 2D elements) and the `/api/fem/solve` route
`fem3d.py`	250	the 3D brick solver and the half-space reference
`docs/`	—	this report, `physics_report` (physics by chapter), `fem_primer`, the FEM plan

1.4 How the pieces fit



There is no build step. The three scripts are ES modules loaded by `index.html`; `sims.js` and `fem.js` register themselves into the same navigation bar, so the FEM tab can be removed by deleting its files (see Chapter 9). The 3D field computes entirely in the browser; only the section plots (and the FEM solves) are sent to the server, because they need to evaluate thousands of points.

1.5 Running the project

The script `run_shape_analytics.sh` starts `uvicorn server:app` on port 8002 using the virtual environment of the sibling project. The server mounts the static files, so the page is served from the same origin as the API.

2 Stress at a point

2.1 The stress tensor and Cauchy's formula (syllabus Chapters 1–2)

Physics. On a cut with unit normal $\mathbf{n}$ the surrounding material exerts a traction $\mathbf{t}_n$. Cauchy's tetrahedron argument shows that the tractions on all planes through a point are determined by those on three coordinate planes, and the linear map is the stress tensor,

$$\mathbf{t}_n = \boldsymbol{\sigma}\,\mathbf{n}, \qquad \sigma_{nn} = \mathbf{n}\cdot\boldsymbol{\sigma}\mathbf{n}, \qquad \tau^2 = \|\mathbf{t}_n\|^2 - \sigma_{nn}^2. \tag{2.1}$$

The last relation splits the traction into its normal and shear parts by Pythagoras.

Code. The route `/api/traction` in `server.py` is a direct transcription. It builds the symmetric matrix, normalises $\mathbf{n}$ (a zero vector falls back to the x axis), applies the matrix and removes the normal component.

```
37 def compute_traction(req: TractionRequest):
38     # Stress tensor (sigma)
39     sigma = np.array([
40         [req.sigma_xx, req.tau_xy, req.tau_zx],
41         [req.tau_xy, req.sigma_yy, req.tau_yz],
42         [req.tau_zx, req.tau_yz, req.sigma_zz]
43     ])
44 
45     # Normal vector (n)
46     n = np.array([req.n_x, req.n_y, req.n_z])
47     norm_n = np.linalg.norm(n)
48     if norm_n > 0:
49         n = n / norm_n # Normalize
50     else:
51         n = np.array([1.0, 0.0, 0.0])
52 
53     # Cauchy's stress theorem: t = sigma * n
54     # Note: Using symmetry, sigma == sigma^T
55     t = np.dot(sigma, n)
56 
57     # Normal component of traction (sigma_n) = t . n
58     sigma_n = np.dot(t, n)
59 
60     # Shear component of traction (tau) vector = t - sigma_n * n
61     tau_vec = t - sigma_n * n
62     tau_mag = np.linalg.norm(tau_vec)
63 
64     return {
65         "traction_vector": t.tolist(),
66         "normal_stress": float(sigma_n),
67         "shear_stress_vector": tau_vec.tolist(),
68         "shear_stress_mag": float(tau_mag)
```

`server.py, lines 37-68`

2.2 Principal stresses and invariants (Chapter 4)

Physics. Principal planes are those on which the traction is purely normal. Maximising σ_{nn} on the unit sphere with a Lagrange multiplier gives the eigenvalue problem $\boldsymbol{\sigma}\mathbf{n} = \lambda\mathbf{n}$, whose characteristic equation is

$$\lambda^3 - I_1\lambda^2 + I_2\lambda - I_3 = 0, \quad I_1 = \operatorname{tr}\boldsymbol{\sigma},\ I_2 = \tfrac{1}{2}[(\operatorname{tr}\boldsymbol{\sigma})^2 - \operatorname{tr}(\boldsymbol{\sigma}^2)],\ I_3 = \det\boldsymbol{\sigma}. \tag{2.2}$$

The roots $\sigma_1 \ge \sigma_2 \ge \sigma_3$ are real because $\boldsymbol{\sigma}$ is symmetric. The largest shear stress acts on planes at 45° to the extreme principal axes: $\tau_{\max} = (\sigma_1 - \sigma_3)/2$. The hydrostatic part $p = I_1/3$ and the deviator $\mathbf{s} = \boldsymbol{\sigma} - p\mathbf{I}$ separate volume change from shape change (Section 4.7 of the course).

Code. Two implementations exist, for two different purposes.

- The server (`/api/principal`) uses `numpy.linalg.eigh`, which also returns the principal *directions*.

```
def compute_principal(req: PrincipalRequest):
    sigma = np.array([
        [req.sigma_xx, req.tau_xy, req.tau_zx],
        [req.tau_xy, req.sigma_yy, req.tau_yz],
        [req.tau_zx, req.tau_yz, req.sigma_zz]
    ])

    # Compute eigenvalues and eigenvectors
    eigenvalues, eigenvectors = np.linalg.eigh(sigma)

    # Sort descending
    idx = eigenvalues.argsort()[::-1]
    eigenvalues = eigenvalues[idx]
    eigenvectors = eigenvectors[:, idx]

    # Maximum shear stress = (sigma_1 - sigma_3) / 2
    tau_max = (eigenvalues[0] - eigenvalues[2]) / 2.0

    # Von Mises stress
    vm = np.sqrt(0.5 * ((eigenvalues[0]-eigenvalues[1])**2 +
                        (eigenvalues[1]-eigenvalues[2])**2 +
                        (eigenvalues[2]-eigenvalues[0])**2))

    return {
        "principal_stresses": eigenvalues.tolist(),
        "principal_directions": eigenvectors.tolist(),
```

server.py, lines 72-97

- The browser needs the principal stresses at tens of thousands of mesh vertices per update, so `physics.js` uses the closed-form (trigonometric) solution of the cubic. With the deviator s_{ij} and its invariants $J_2 = \frac{1}{2}s_{ij}s_{ij}$ and $J_3 = \det \mathbf{s}$, the roots are

$$\sigma_k = p + 2\sqrt{J_2/3}\,\cos\Big(\phi - \tfrac{2\pi k}{3}\Big), \quad \phi = \tfrac{1}{3}\arccos\Big(\frac{3\sqrt{3}\,J_3}{2J_2^{3/2}}\Big), \quad k = 0, 1, 2. \tag{2.3}$$

```
export function principalStresses(t) {
    const [a, b, c, d, f, e] = t; // d = tauxy, f = tauxz, e = tauyz
    const p = (a + b + c) / 3;
    const sa = a - p, sb = b - p, sc = c - p;
    const j2 = 0.5 * (sa * sa + sb * sb + sc * sc) + d * d + e * e + f * f;
    // Hydrostatic state: the deviator is zero to rounding. The test is relative to the stress scale, so a
    // genuinely tiny stress state (e.g. far from a load) is still resolved.
    const scale2 = a * a + b * b + c * c + 2 * (d * d + e * e + f * f);
    if (j2 <= 1e-24 * scale2) return [p, p, p];
    const j3 = sa * (sb * sc - e * e) - d * (d * sc - e * f) + f * (d * e - sb * f);
    const r = 2 * Math.sqrt(j2 / 3);
    const phi = Math.acos(Math.max(-1, Math.min(1, (3 * Math.sqrt(3) * j3) / (2 * j2 ** 1.5)))) / 3;
    return [0, 1, 2].map((k) => p + r * Math.cos(phi - (2 * Math.PI * k) / 3)).sort((m, n) => n - m);
}

export const vonMises = (t) => Math.sqrt(0.5 * ((t[0] - t[1]) ** 2 + (t[1] - t[2]) ** 2 + (t[2] - t[0]) ** 2 + 6 * (t[3] ** 2 + t[4]
↪  ** 2 + t[5] ** 2)));
```

physics.js, lines 75-90

Note. Three numerical details matter. (i) The argument of arccos is clamped to $[-1, 1]$, because rounding can push it just outside when two roots coincide. (ii) A stress state whose deviator is zero to rounding (hydrostatic) returns three equal roots; the test is *relative* to the stress scale, so very small stress states are not mistaken for hydrostatic. (iii) Results are sorted into descending order. The invariants of random tensors across twelve decades of magnitude were checked against the roots (Chapter 8).

2.3 Equivalent stresses

Physics. A failure theory reduces the stress tensor to one number to compare with a tension test (Chapter 11). The two used throughout the app are

$$\sigma_{\text{Tresca}} = \sigma_1 - \sigma_3, \qquad \sigma_{vM} = \sqrt{\tfrac{1}{2}\big[(\sigma_1-\sigma_2)^2 + (\sigma_2-\sigma_3)^2 + (\sigma_3-\sigma_1)^2\big]}. \tag{2.4}$$

The von Mises stress is $\sqrt{3J_2}$: it measures only the deviatoric (shape-changing) part, which is why it vanishes in hydrostatic compression. In terms of components it equals $\sqrt{\frac{1}{2}[(\sigma_{xx}-\sigma_{yy})^2 + \ldots] + 3(\tau_{xy}^2 + \tau_{yz}^2 + \tau_{zx}^2)}$.

Code. `physics.js` contains the component form (`vonMises`); the simulators use the principal form.

```
90 export const vonMises = (t) => Math.sqrt(0.5 * ((t[0] - t[1]) ** 2 + (t[1] - t[2]) ** 2 + (t[2] - t[0]) ** 2 + 6 * (t[3] ** 2 + t[4]
   ↪  ** 2 + t[5] ** 2)));
```

`physics.js, lines 90-90`

2.4 Mohr's circle simulator (Chapters 2 and 4)

Physics. Rotating the element about z by θ changes the face stresses by the transformation law

$$\sigma_{x'} = \tfrac{\sigma_x+\sigma_y}{2} + \tfrac{\sigma_x-\sigma_y}{2}\cos 2\theta + \tau_{xy}\sin 2\theta, \tag{2.5}$$

$$\tau_{x'y'} = -\tfrac{\sigma_x-\sigma_y}{2}\sin 2\theta + \tau_{xy}\cos 2\theta, \tag{2.6}$$

so the point $(\sigma_{x'}, \tau_{x'y'})$ moves around a circle of radius $\sqrt{(\frac{\sigma_x-\sigma_y}{2})^2+\tau_{xy}^2}$ about $(\sigma_x+\sigma_y)/2$, at twice the rotation angle. In three dimensions there are three such circles through the principal stresses, and all admissible states lie between them.

Code. `mohr.compute` in `sims.js` applies the transformation law, calls the three-dimensional solver `eig3` (the same trigonometric solution) and draws the three circles. The out-of-plane shears τ_{yz}, τ_{zx} enter the eigenproblem but not the in-plane rotation.

```
84         const t2 = rad(2 * theta), avg = (sx + sy) / 2, d = (sx - sy) / 2;
85         const sxp = avg + d * Math.cos(t2) + txy * Math.sin(t2);
86         const syp = avg - d * Math.cos(t2) - txy * Math.sin(t2);
87         const txyp = -d * Math.sin(t2) + txy * Math.cos(t2);
88         const e = eig3([[sx, txy, tzx], [txy, sy, tyz], [tzx, tyz, sz]]);
89         const { p1, p2, p3 } = e;
90         const thetaP = 0.5 * deg(Math.atan2(2 * txy, sx - sy));
91         const tmax = (p1 - p3) / 2;
92         const I1 = sx + sy + sz;
93         const I2 = sx * sy + sy * sz + sz * sx - txy ** 2 - tyz ** 2 - tzx ** 2;
94         const I3 = p1 * p2 * p3;
95         const vm = Math.sqrt(0.5 * ((p1 - p2) ** 2 + (p2 - p3) ** 2 + (p3 - p1) ** 2));
96         const tresca = p1 - p3;
97         const eq = v.theory === 'vm' ? vm : tresca;
98         const sf = eq > 1e-9 ? syield / eq : Infinity;
99         const circles = [[p1, p3, 'sigma1-sigma3'], [p1, p2, 'sigma1-sigma2'], [p2, p3, 'sigma2-sigma3']].map(([a, b, n], i) => {
```

`sims.js, lines 84-99`

Note. The safety factor is $\sigma_y/\sigma_{\mathrm{eq}}$ for the chosen theory. When $\sigma_{\mathrm{eq}} = 0$ (a hydrostatic state) it is infinite and the interface shows ∞. The simulator accepts *symmetric* stress matrices only: moment equilibrium (Chapter 3.2) forces $\tau_{ij} = \tau_{ji}$.

3 Half-space elasticity: the equations behind the 3D field

3.1 Governing equations (Chapters 3, 5, 6)

Physics. A linear elastic body in static equilibrium satisfies

$$\nabla\cdot\boldsymbol{\sigma} + \mathbf{b} = \mathbf{0}, \qquad \boldsymbol{\epsilon} = \tfrac{1}{2}(\nabla\mathbf{u} + \nabla\mathbf{u}^T), \qquad \boldsymbol{\sigma} = \lambda\,\mathrm{tr}(\boldsymbol{\epsilon})\mathbf{I} + 2G\boldsymbol{\epsilon}, \tag{3.1}$$

with $\lambda = E\nu/[(1+\nu)(1-2\nu)]$ and $G = E/[2(1+\nu)]$ (and $K = E/[3(1-2\nu)]$ for the bulk modulus, also shown in the app). Combined, these give the Navier equations for the displacement. They have closed-form solutions only for special geometries. The simplest useful one is the *half-space*: the body fills $z > 0$, the surface $z = 0$ is free except where a load acts, and the body is infinitely deep and wide.

The app uses this solution because it is the textbook model of "a load pressed on the surface of a big solid", and because loads from a painted area can be built by superposition. The price is the infinite body: the half-space solution does not give zero traction on the sides and base of a finite body. Chapter 7 quantifies that error with a finite element model.

3.2 Normal point load: Boussinesq

Physics. A force P pressing into the surface at the origin produces, at depth z and radius r ($R^2 = r^2 + z^2$), the stress components (tension positive)

$$\sigma_z = -\frac{3Pz^3}{2\pi R^5}, \qquad \tau_{rz} = -\frac{3Prz^2}{2\pi R^5}, \tag{3.2}$$

$$\sigma_r = \frac{P}{2\pi}\Big[\frac{1-2\nu}{R(R+z)} - \frac{3r^2 z}{R^5}\Big], \qquad \sigma_\theta = \frac{P(1-2\nu)}{2\pi}\Big[\frac{z}{R^3} - \frac{1}{R(R+z)}\Big]. \tag{3.3}$$

The hoop stress σ_θ is not zero; leaving it out would give the wrong von Mises stress and strain. Three tests of correctness are used: (i) $\nabla\cdot\boldsymbol{\sigma} = \mathbf{0}$; (ii) the surface $z = 0$ is stress-free outside the load; (iii) the vertical force through every horizontal plane equals the load, $\int_0^\infty \sigma_z\, 2\pi r\, dr = -P$.

Code. `boussinesqTensor` evaluates these in cylindrical form and rotates to Cartesian components with the azimuth $(\cos\varphi, \sin\varphi) = (x/r, y/r)$. On the axis ($r = 0$) the azimuth is undefined, but $\sigma_r = \sigma_\theta$ there, so the code returns the average and zero shear.

```
44 export function boussinesqTensor(x, y, z, Pz, a, nu) {
45     const ze = z + a;
46     const r2 = x * x + y * y;
47     const R = Math.sqrt(r2 + ze * ze);
48     const R3 = R * R * R, R5 = R3 * R * R;
49     const C = Pz / (2 * Math.PI);
50     const sr = C * ((1 - 2 * nu) / (R * (R + ze)) - (3 * r2 * ze) / R5);
51     const st = C * (1 - 2 * nu) * (ze / R3 - 1 / (R * (R + ze)));
52     const sz = (-3 * Pz * ze * ze * ze) / (2 * Math.PI * R5);
53     const trz = (-3 * Pz * ze * ze) / (2 * Math.PI * R5); // times r, split below
54     if (r2 < 1e-16) return [(sr + st) / 2, (sr + st) / 2, sz, 0, 0, 0];
55     const r = Math.sqrt(r2), c = x / r, s = y / r;
56     return [sr * c * c + st * s * s, sr * s * s + st * c * c, sz, (sr - st) * c * s, trz * r * c, trz * r * s];
57 }
```

`physics.js, lines 44-57`

3.3 Tangential point load: Cerruti

Physics. A force along the surface produces a different field, known in closed form as a displacement (Cerruti). The program uses exactly the chain of the course, Chapters 5 and 6: take the displacement, form the strain as the symmetric gradient, and apply Hooke's law. The displacement of a point load P_z also uses the Boussinesq form, for example $u_z = \frac{P(1+\nu)}{2\pi E R}\big[2(1-\nu) + z^2/R^2\big]$, and at the surface this reduces to the familiar $u_z = P(1-\nu^2)/(\pi E r)$.

```
    if (Pz !== 0) { // Boussinesq
        const C = (Pz * (1 + nu)) / (2 * Math.PI * E * R);
        uz += C * (2 * (1 - nu) + (ze * ze) / (R * R));
        if (r2 > 0) {
            const r = Math.sqrt(r2);
            // u_r = P(1+nu)/(2piE r) [ r^2z/R^3 + (1-2nu)(z/R - 1) ]
            const ur = (Pz * (1 + nu)) / (2 * Math.PI * E * r) * ((r2 * ze) / (R * R * R) + (1 - 2 * nu) * (ze / R - 1));
            ux += ur * (x / r);
            uy += ur * (y / r);
```

physics.js, lines 17-25

Code. `tangentialTensor` differentiates the displacement field numerically with a central step $h = 10^{-4}\sqrt{x^2 + y^2 + z_e^2}$ and then applies Hooke's law. Using the displacement as the primary object guarantees that the resulting strain satisfies compatibility (Chapter 5.9), and only the displacement formula (which is shorter and easier to verify) has to be typed in.

```
export function tangentialTensor(x, y, z, Px, Py, a, nu, E) {
    const ze = z + a;
    const h = 1e-4 * Math.sqrt(x * x + y * y + ze * ze);
    const u = (xx, yy, zz) => { const d = halfSpaceDisplacement(xx, yy, zz, Px, Py, 0, a, nu, E); return [d.ux, d.uy, d.uz]; };
    const dx = [u(x + h, y, z), u(x - h, y, z)], dy = [u(x, y + h, z), u(x, y - h, z)], dz = [u(x, y, z + h), u(x, y, z - h)];
    const g = (pair, i) => (pair[0][i] - pair[1][i]) / (2 * h); // du_i/d(that axis)
    const exx = g(dx, 0), eyy = g(dy, 1), ezz = g(dz, 2);
    const exy = 0.5 * (g(dy, 0) + g(dx, 1)), exz = 0.5 * (g(dz, 0) + g(dx, 2)), eyz = 0.5 * (g(dz, 1) + g(dy, 2));
    const nuL = Math.min(nu, 0.499); // lambda -> infinity as nu -> 0.5 (incompressible); cap keeps the stress finite
    const lam = (E * nuL) / ((1 + nuL) * (1 - 2 * nuL)), G = E / (2 * (1 + nu)), tr = exx + eyy + ezz;
    return [lam * tr + 2 * G * exx, lam * tr + 2 * G * eyy, lam * tr + 2 * G * ezz, 2 * G * exy, 2 * G * exz, 2 * G * eyz];
}

// Principal stresses sigma1 >= sigma2 >= sigma3 of [sigmaxx, sigmayy, sigmazz, tauxy, tauxz, tauyz] (Ch. 4: eigenvalues of the stress
↪  matrix).
export function principalStresses(t) {
    const [a, b, c, d, f, e] = t; // d = tauxy, f = tauxz, e = tauyz
```

physics.js, lines 61-76

Note. $\lambda \to \infty$ as $\nu \to 0.5$ (incompressible), so ν is capped at 0.499 in this formula; the 3D tab's materials never reach this value (rubber is 0.49).

3.4 Equivalent depth: removing the singularity

Under a point load the half-space stress is infinite. A painted patch, however, is a pressure distributed over an area, not a point. The code represents the patch by point loads at its mesh vertices and shifts each source by an equivalent depth, $z_e = z + a$ (the `a` argument). The offset is chosen from the geometry of the patch (average distance of the painted vertices from the load point, divided by the square root of their number, times 2.5) so that neighbouring point loads overlap smoothly. It is a numerical smoothing, not a physical parameter; a finite element model of the same patch (Section 7.5) shows how well it behaves away from the surface.

3.5 Superposition (Chapter 10.1)

Because the governing equations are linear, the field of several loads is the sum of the fields of each. This is used twice: over the painted vertices of one patch, and over *saved load cases*. The sum must be taken on the **tensors**. Earlier versions of the code took the maximum of scalars, which is wrong because von Mises stress and principal stresses are not linear in the tensor. The next chapter shows where the sum is made.

4 The 3D stress field pipeline

4.1 From geometry to loads

Physics. A pressure p over a patch of area A is statically equivalent to a force $P = pA$ along the load direction. The patch area is the sum of the areas of the mesh triangles it covers, not the number of painted vertices, so the force does not depend on how finely the mesh is sampled.

Code. The body is a three.js mesh (box, cylinder or sphere) whose original vertex positions are kept in `basePositions`. `computeVertexAreas` gives each vertex a third of the area of every triangle it belongs to; `paintedAreaOf` then sums these for a set of painted vertices.

```
384 function computeVertexAreas() {
385     const pos = geometry.attributes.position.array;
386     const idx = geometry.index ? geometry.index.array : null;
387     const nTri = idx ? idx.length / 3 : geometry.attributes.position.count / 3;
388     vertexArea = new Float64Array(geometry.attributes.position.count);
389     surfaceArea = 0;
390     for (let t = 0; t < nTri; t++) {
391         const i0 = idx ? idx[3*t] : 3*t, i1 = idx ? idx[3*t+1] : 3*t+1, i2 = idx ? idx[3*t+2] : 3*t+2;
392         const ux = pos[i1*3]-pos[i0*3], uy = pos[i1*3+1]-pos[i0*3+1], uz = pos[i1*3+2]-pos[i0*3+2];
393         const vx = pos[i2*3]-pos[i0*3], vy = pos[i2*3+1]-pos[i0*3+1], vz = pos[i2*3+2]-pos[i0*3+2];
394         const A = 0.5 * Math.hypot(uy*vz-uz*vy, uz*vx-ux*vz, ux*vy-uy*vx);
395         vertexArea[i0] += A/3; vertexArea[i1] += A/3; vertexArea[i2] += A/3;
396         surfaceArea += A;
397     }
398 }
399 function paintedAreaOf(indices) {
400     let s = 0;
401     indices.forEach(i => { s += vertexArea[i]; });
402     return Math.max(1e-6, s);
403 }
```

app.js, lines 384-403

The load pressure typed in kN/m^2 becomes a force in newtons as `P = load.force * area * 1000`, with `area` in m^2.

4.2 A local frame for each load

Physics. Both half-space solutions are written in a frame whose z axis points into the body. For a load on a face with outward normal $\mathbf{n}_s$ the frame is $\mathbf{n}_z = -\mathbf{n}_s$, with any two perpendicular tangent vectors $\mathbf{n}_x, \mathbf{n}_y$. A load direction $\mathbf{f}$ is split into a normal part $f_z = \mathbf{f} \cdot \mathbf{n}_z$ and a tangential part with components (f_x, f_y) in the face plane.

```
454         const nz = load.loadNormal.clone().multiplyScalar(-1);
455         const nx = new THREE.Vector3(1, 0, 0);
456         if (Math.abs(nz.x) > 0.9) nx.set(0, 1, 0);
457         nx.cross(nz).normalize();
458         const ny = new THREE.Vector3().crossVectors(nz, nx).normalize();
459
460         const forceDir = nz.clone();
461         if (load.tractionType === 'arbitrary') forceDir.set(load.vecX, load.vecY, load.vecZ).normalize();
462         const fz = forceDir.dot(nz);
463         const fT = forceDir.clone().sub(nz.clone().multiplyScalar(fz));
464         const fx = fT.dot(nx), fy = fT.dot(ny);
465
466         // load.force is a PRESSURE in kN/m^2; total force in N = pressure * painted area * 1000.
467         const area = paintedAreaOf(load.indices);
468         const coverage = area / surfaceArea;
469         const P = load.force * area * 1000;
470         const Px = P * fx, Py = P * fy, Pz = P * fz;
471         const pressure = P / area;
472
473         let avgDist = 0;
474         load.indices.forEach(idx => {
475             avgDist += Math.hypot(basePositions[idx*3]-load.loadPoint.x, basePositions[idx*3+1]-load.loadPoint.y,
            ↪  basePositions[idx*3+2]-load.loadPoint.z);
```

app.js, lines 454-475

Note. The server builds its frame with the opposite sign convention for the tangent vectors ($\mathbf{n}_x \to -\mathbf{n}_x$, $\mathbf{n}_y \to -\mathbf{n}_y$). The two frames are rotated by 180° about $\mathbf{n}_z$ relative to each other, so the global tensor is identical; this was confirmed numerically by comparing the browser and the server on four load orientations.

4.3 Superposition and rotation to global axes

Physics. The tensor of one point source is computed in the local frame and rotated to global axes by $\boldsymbol{\sigma}_g = \mathbf{R}\,\boldsymbol{\sigma}_l\,\mathbf{R}^T$, where the columns of $\mathbf{R}$ are $\mathbf{n}_x, \mathbf{n}_y, \mathbf{n}_z$. Rotating the *tensor*, not just its components, is Chapter 2.5 (transformation of the stress matrix).

```
505     const rot = (info, t) => { // local tensor -> global: R T R^T with R = [nx ny nz]
506         const a = [info.nx, info.ny, info.nz], out = [0, 0, 0, 0, 0, 0];
507         const L = [[t[0], t[3], t[4]], [t[3], t[1], t[5]], [t[4], t[5], t[2]]];
508         const pairs = [[0,0],[1,1],[2,2],[0,1],[0,2],[1,2]];
509         pairs.forEach(([p, q], k) => {
510             let s = 0;
511             for (let i = 0; i < 3; i++) for (let j = 0; j < 3; j++) {
512                 s += L[i][j] * a[i].getComponent(p) * a[j].getComponent(q);
513             }
514             out[k] = s;
515         });
516         return out;
```

app.js, lines 505-516

Code. For every vertex, the loop below adds the tensors from every point of every load into one array `S`. Only after the loop are the principal stresses, von Mises stress and strain computed from `S`.

```
526         for (const info of infos) {
527             const { nx, ny, nz } = info;
528             const rc = vertexPos.clone().sub(info.load.loadPoint);
529             const xc = rc.dot(nx), yc = rc.dot(ny), zc = Math.max(0.01, rc.dot(nz));
530             const dv = halfSpaceDisplacement(xc, yc, zc, info.Px, info.Py, info.Pz, info.effectivePatch, nu, E);
531             const dispBouss = nx.clone().multiplyScalar(dv.ux).add(ny.clone().multiplyScalar(dv.uy)).add(nz.clone().multiplyScalar(dv
                ↪  .uz)).sub(info.dispCorr);
532
533             if (info.full) {
534                 const dispHydro = new THREE.Vector3(bx, by, bz).multiplyScalar(info.eps_hydro);
535                 dispHydro.y -= minY * info.eps_hydro;
536                 totalDisp.add(dispHydro.multiplyScalar(scale));
537                 S[0] -= info.pressure; S[1] -= info.pressure; S[2] -= info.pressure;
538                 continue;
539             }
540             totalDisp.add(dispBouss.multiplyScalar(scale));
541
542             // Stress by superposition of point-force solutions over the painted points.
543             const local = [0, 0, 0, 0, 0, 0];
544             const hasTan = info.Px !== 0 || info.Py !== 0;
545             for (const sp of info.sp) {
546                 const rel = vertexPos.clone().sub(sp);
547                 const x = rel.dot(nx), y = rel.dot(ny), z = Math.max(0.01, rel.dot(nz));
548                 const b = boussinesqTensor(x, y, z, info.Pz * info.dP, info.patchLocal, nu);
549                 for (let k = 0; k < 6; k++) local[k] += b[k];
550                 if (hasTan) {
551                     const c = tangentialTensor(x, y, z, info.Px * info.dP, info.Py * info.dP, info.patchLocal, nu, E);
552                     for (let k = 0; k < 6; k++) local[k] += c[k];
553                 }
554             }
555             const g = rot(info, local);
556             for (let k = 0; k < 6; k++) S[k] += g[k];
557         }
558
559         displacements[i] = totalDisp;
560         const mag = totalDisp.length();
561         if (mag > maxDispMag) maxDispMag = mag;
```

app.js, lines 526-561

4.4 Full-surface pressure

Physics. If a normal pressure p acts on the *entire* boundary, the traction is $\mathbf{t} = -p\mathbf{n}$ on every face, and the uniform state $\boldsymbol{\sigma} = -p\mathbf{I}$ satisfies equilibrium and the boundary condition everywhere (Chapter 2). The body is compressed hydrostatically: all principal stresses are $-p$ and the von Mises stress is zero. The half-space solution is the wrong model for this case.

Code. The program switches to the hydrostatic state only when the normal pressure covers at least 99% of the surface *area*. Below that, the field is the half-space solution. The volumetric strain $\epsilon_V = -p/K$ shrinks the displayed body and changes the density to $\rho/(1+\epsilon_V)$.

```
483         const full = load.tractionType === 'normal' && coverage >= 0.99;
484         const eps_hydro = full ? -pressure / (3 * K) : 0;
485         eps_hydro_total += eps_hydro;
```

app.js, lines 483-485

4.5 Quantities displayed

Physics. Four maps are available. With σ_1, σ_3 the extreme principal stresses of the *summed* tensor:

Map	Definition
Normal stress	$\max(\lvert\sigma_1\rvert, \lvert\sigma_3\rvert)$
Max shear	$\tau_{\max} = (\sigma_1 - \sigma_3)/2$
Von Mises	$\sqrt{\frac{1}{2}[(\sigma_{xx}-\sigma_{yy})^2 + \dots] + 3(\tau^2)}$
Strain	$\lVert\boldsymbol{\epsilon}\rVert$ with $\boldsymbol{\epsilon} = [(1+\nu)\boldsymbol{\sigma} - \nu\,\mathrm{tr}(\boldsymbol{\sigma})\mathbf{I}]/E$ (inverse Hooke)

Poisson's ratio is chosen per material (steel 0.30, aluminium 0.33, concrete 0.20, polymer 0.38, rubber 0.49).

```
563         // Ch. 4: extreme normal stress and tau_max = (s1 - s3)/2 from the principal stresses.
564         const [s1, , s3] = principalStresses(S);
565         const sigN = Math.max(Math.abs(s1), Math.abs(s3)) / 1e6;
566         const tau = (s1 - s3) / 2 / 1e6;
567         const vm = vonMises(S) / 1e6;
568         const eps = strainNorm(S, nu, E) * 1e6; // microstrain
569
570         let val = 0;
571         if (state.mapType === 'sigma_n') val = sigN;
572         if (state.mapType === 'tau_max') val = tau;
```

app.js, lines 563-572

Code. Colour is a normalised stress ramp: blue (low) through paper to vermilion (high), identical to the ramp used by the section plots and the FEM tab (`stressRamp`). Displacements are the Boussinesq–Cerruti displacements summed over the loads, measured relative to the bottom centre of the body, then multiplied by a user-adjustable exaggeration factor. The factor does not depend on E, so the ratio of displacements between two materials is physical. If the largest displacement would exceed a safe visual limit, *all* displacements are scaled down by the same factor so the shape is not distorted.

Note. Strain above 1% ($10\,000\,\mu\epsilon$) is outside small-strain theory. Rubber at high load can exceed 100%; the strain readout then carries the label “beyond small-strain theory”.

5 Section analytics

5.1 What a section shows

The drawer under the 3D viewport shows the stress on a plane cut through the body. The user chooses the plane (xy, xz or yz) and its position. This is useful because the 3D map shows only the *surface* of the body while the interior stress is different: under a surface load the stress decays with depth and shear stress appears below the surface.

5.2 Server-side evaluation

Physics. The same half-space tensor of Chapter 3 is evaluated on a regular grid of points in the cut plane, summed over all loads, and converted to the four displayed measures. The grid is the bounding rectangle of the body, so points that fall outside a cylinder or a sphere must be removed.

Code. `compute_section` in `server.py` repeats the frame construction and superposition of the browser, vectorised over the grid with numpy. It then forms the stress array, takes its eigenvalues for all points at once with `numpy.linalg.eigvalsh`, and evaluates the four fields.

```
300     # Same measures as the 3D view (Ch. 4, 5, 11): extreme normal stress, tau_max = (s1-s3)/2, von Mises, |strain|.
301     p = np.linalg.eigvalsh(S)               # ascending: s3, s2, s1
302     s1, s3 = p[:, 2], p[:, 0]
303     vm = np.sqrt(0.5 * ((s_xx_tot - s_yy_tot)**2 + (s_yy_tot - s_zz_tot)**2 + (s_zz_tot - s_xx_tot)**2
304                         + 6 * (t_xy_tot**2 + t_yz_tot**2 + t_xz_tot**2)))
305     k = (1 + nu) / E
306     m = nu * (s_xx_tot + s_yy_tot + s_zz_tot) / E
307     eps = np.sqrt((k*s_xx_tot - m)**2 + (k*s_yy_tot - m)**2 + (k*s_zz_tot - m)**2
308                   + 2 * k**2 * (t_xy_tot**2 + t_yz_tot**2 + t_xz_tot**2))
309     fields = {
310         "sigma_n": np.maximum(np.abs(s1), np.abs(s3)) / 1e6,
311         "tau_max": (s1 - s3) / 2 / 1e6,
312         "von_mises": vm / 1e6,
313         "strain_mag": eps * 1e6,            # microstrain
314     }
```

server.py, lines 300-314

Points outside the body are replaced by `null`, which Plotly draws as blank. The mask follows the true shape (box, cylinder with its axis along y, sphere):

```
316     # Keep only points inside the body; the grid is its bounding rectangle. Outside -> null (blank in the plot).
317     hx, hy, hz = req.dim_x / 2, req.dim_y / 2, req.dim_z / 2
318     tol = 1e-9
319     if req.shape == 'cylinder':
320         inside = (x**2 + z**2 <= hx**2 + tol) & (np.abs(y) <= hy + tol)
321     elif req.shape == 'sphere':
322         inside = x**2 + y**2 + z**2 <= hx**2 + tol
323     else:
324         inside = (np.abs(x) <= hx + tol) & (np.abs(y) <= hy + tol) & (np.abs(z) <= hz + tol)
325
326     def grid(v):
```

server.py, lines 316-326

5.3 Front end

`app.js` (`updateAnalyticsPlots`) assembles the request from the current geometry, material and loads. It clamps the slice-position slider to the extent of the body along the slice axis, so a plane can never lie outside it. Each request carries a sequence number; a reply that arrives after a newer request was sent is ignored, so dragging the slider cannot display a stale picture. If the server is unreachable the plot shows a message

instead of an old result. The section uses the same map selector as the 3D view, so the colour scale and the quantity are consistent between the two.

Note. Resolution is clamped to 2–400 on the server (a grid needs two points and the cost grows with the square), and the force direction is normalised there as a defence against inconsistent requests.

6 The analytical simulators

6.1 The framework in `sims.js`

Physics. Each simulator is a function from a few input numbers to a set of results, so the interface only needs to know how to draw the inputs and the outputs. There is no physics in the framework.

Code. Every simulator is a plain object with a fixed shape. The framework builds sliders and selectors from `inputs`, calls `compute(v)` whenever an input changes (once per animation frame), and draws what it returns.

```
{ id, title, chapters, blurb,
  inputs: [ { k, label, unit, min, max, step, value }  |  { k, label, type:'select', options:[[val,label],...], value } ],
  compute(v) -> { status:{ok,label,summary},          // the coloured banner
                  readouts:[{label,value,unit}],      // the title-block numbers
                  charts:[{id,title,x,y,traces,primary,square}],
                  diagram?, view3d?, equations:[...] },
  significance }
```

- **Rendering** (`render`): updates the status banner, the SVG diagram, the readout cards, and one Plotly chart per entry of `charts`. Theme colours are resolved from the CSS variables, and a `MutationObserver` on the `data-theme` attribute redraws everything when the user switches between light and dark.
- **A/B comparison**: "Save state A" stores the current result. Later results are drawn with the stored traces dotted and each readout shows the stored value, so the effect of changing one parameter is visible directly.
- **Export**: CSV of inputs, readouts and chart data; SVG of the diagram; a design-specification sheet; and a lab report in HTML. Each chart also has Plotly's own PNG button.
- **Routing**: the module id is kept in the URL hash (`#beam`), so a link opens a particular simulator.
- **Extension**: `registerModule` lets another file add a tab with its own page (the FEM tab does this) without `sims.js` knowing about it.

```
768 export function registerModule(def) {
769     extras.set(def.id, def);
770     const b = document.createElement('button');
771     b.type = 'button'; b.dataset.module = def.id; b.title = def.chapters; b.textContent = def.title;
772     $('module-nav').appendChild(b);
773     if (INITIAL_HASH === def.id) showModule(def.id);
774 }
775
776 export function showModule(id) {
```

sims.js, lines 768–776

6.2 Beams (Chapters 8 and 9)

Physics. For a beam in static equilibrium the shear force and bending moment satisfy $dV/dx = -w$ and $dM/dx = V$. In Euler–Bernoulli theory (plane sections stay plane and normal to the axis) the curvature is $v'' = M/EI$, the bending stress is $\sigma = My/I$ and the shear stress is $\tau = VQ/(Ib)$ with Q the first moment of the area above the point. Timoshenko theory lets the section rotate independently of the axis, which adds a shear deflection $v_s' = V/(k_s GA)$; $k_s = 10(1+\nu)/(12+11\nu)$ for a rectangle and $6(1+\nu)/(7+6\nu)$ for a solid circle. The extra deflection is large only for short, deep beams: for a cantilever with a tip load $\delta = PL^3/3EI + PL/(k_s GA)$.

Code. The support reactions follow from statics for the chosen case (simply supported or cantilever; point load, uniform load and an applied moment, combined by superposition). $V(x)$ and $M(x)$ are evaluated on 401 points. The deflection is obtained by integrating M/EI twice with the trapezoidal rule (`cumtrapz`) and then imposing the boundary conditions: zero deflection at both supports for the simply supported beam (subtracting the straight line that closes the end value), and zero slope and deflection at the wall for the cantilever.

```
191         const slope0 = cumtrapz(x, Mx.map((q) => q / EI));
192         const raw = cumtrapz(x, slope0);
193         let vy;
194         if (v.type === 'ss') vy = raw.map((q, i) => q - (raw.at(-1) * x[i]) / L);
195         else vy = raw;
196         let shearDefl = x.map(() => 0);
197         if (v.theory === 'tbt') {
198             const ys = cumtrapz(x, Vx.map((q) => q / (sec.ks * G * sec.A)));
199             shearDefl = v.type === 'ss' ? ys.map((q, i) => q - (ys.at(-1) * x[i]) / L) : ys;
200         }
```

sims.js, lines 191-200

The shear stress through the depth uses a numerical integral for $Q(y)$, so it is correct for the rectangle, the circle and the tube without separate formulas:

```
207         const Q = ys.map((y) => { let q = 0; const steps = 60; for (let i = 0; i < steps; i++) { const yy = y + ((sec.c - y) * (i +
            ↪  0.5)) / steps; q += yy * sec.width(yy) * ((sec.c - y) / steps); } return q; });
208         const tau = ys.map((y, i) => { const bw = sec.width(y); return bw > 1e-6 ? (Vmax * Q[i]) / (sec.I * bw) : 0; });
```

sims.js, lines 207-208

Note. For the rectangle the maximum is $1.5\,V/A$ and for the circle $4V/3A$; both were reproduced to better than 0.5%. A hollow section with the inner diameter larger than the outer is corrected to 0.8 of the outer diameter instead of failing.

6.3 Shaft torsion (Chapter 7)

Physics. For a circular shaft, plane sections stay plane and rotate rigidly, so the shear strain grows linearly with radius, $\gamma = r\,d\phi/dx$, and Hooke's law gives $\tau = Gr\,d\phi/dx$. Equilibrium of the torque gives

$$\tau(r) = \frac{Tr}{J}, \qquad \phi = \frac{TL}{GJ}, \qquad J = \frac{\pi}{32}(D^4 - d^4), \qquad G = \frac{E}{2(1+\nu)}. \tag{6.1}$$

A hollow shaft is nearly as strong as a solid one of the same diameter because the material near the axis carries little stress.

```
258         const J = (Math.PI * (v.D ** 4 - v.d ** 4)) / 32, T = v.T * 1000, L = v.L * 1000;
259         const tmax = (T * (v.D / 2)) / J, tmin = (T * (v.d / 2)) / J;
260         const phi = (T * L) / (G * J), k = (G * J) / L / 1000; // N*m/rad
261         const A = (Math.PI * (v.D ** 2 - v.d ** 2)) / 4;
```

sims.js, lines 258-261

Code. The simulator also produces a three-dimensional view (`view3d` with `draw3d` and `update3d` in `sims.js`): a three.js shaft with a coarse wireframe, an end face coloured by $\tau \propto r$, a fixed wall, and an axial line that follows the twist. The twist is exaggerated when it is small (the note under the view gives the factor).

6.4 Axial loading (Chapters 6 and 7)

Physics. A bar of several segments, each with its own length L_i, area A_i and load P_i, carries internal forces N_i found by cutting the bar. If one end is free, N_i is the sum of the loads beyond the cut; the elongation of a segment is $N_i L_i/(EA_i) + \alpha \Delta T\, L_i$. If both ends are fixed the problem is statically indeterminate: choose the wall reaction X as the unknown (the redundant), write $N_i = N_i^{\text{free}} + X$, and impose compatibility that the total elongation is zero. This gives

$$X = -\frac{\sum_i N_i^{\text{free}} L_i/(EA_i) + \alpha \Delta T \sum_i L_i}{\sum_i L_i/(EA_i)}. \tag{6.2}$$

```
302         const Nfree = seg.map((_, i) => seg.slice(i).reduce((s, q) => s + q.P, 0));
```

sims.js, lines 302-302

```
304         if (v.bc === 'fixed') {
305             const num = seg.reduce((s, q, i) => s + (Nfree[i] * q.L) / (E * q.A), 0) + al * v.dT * seg.reduce((s, q) => s + q.L, 0);
306             const den = seg.reduce((s, q) => s + q.L / (E * q.A), 0);
307             X = -num / den;
308         }
309         const N = Nfree.map((q) => q + X);
```

sims.js, lines 304-309

Note. With a uniform temperature rise and no load on a fixed bar the formula gives $N = -EA\alpha\Delta T$: -120 MPa for steel at 50°C. The net elongation of a fixed–fixed bar is zero to rounding, whatever the loads.

6.5 Pressure vessels (Chapter 7)

Physics. Thin wall: equilibrium of half a cylinder gives a hoop stress $\sigma_\theta = pr/t$ and, with closed ends, an axial stress $pr/2t$; for a sphere $\sigma_\theta = pr/2t$. Thick wall: for an axisymmetric field the equilibrium $d\sigma_r/dr + (\sigma_r - \sigma_\theta)/r = 0$ combined with compatibility gives the Lamé solution,

$$\text{cylinder:}\quad \sigma_r = A - \frac{B}{r^2},\ \sigma_\theta = A + \frac{B}{r^2},\ \sigma_z = A, \qquad A = \frac{p_i a^2 - p_o b^2}{b^2 - a^2},\ B = \frac{(p_i - p_o)a^2 b^2}{b^2 - a^2}, \tag{6.3}$$

$$\text{sphere:}\quad \sigma_r = A - \frac{2B}{r^3},\ \sigma_\theta = \sigma_\phi = A + \frac{B}{r^3}, \qquad A = \frac{p_i a^3 - p_o b^3}{b^3 - a^3},\ B = \frac{(p_i - p_o)a^3 b^3}{2(b^3 - a^3)}. \tag{6.4}$$

The hoop stress is greatest at the inner wall. The radial growth is $u = \frac{r}{E}[\sigma_\theta - \nu(\sigma_r + \sigma_z)]$.

```
366         if (cyl) {
367             A = (pi * a * a - po * b * b) / (b * b - a * a); B = ((pi - po) * a * a * b * b) / (b * b - a * a);
368             sr = r.map((q) => A - B / (q * q)); st = r.map((q) => A + B / (q * q)); sz = r.map(() => A);
369             u = r.map((q, i) => (q / E) * (st[i] - nu * (sr[i] + sz[i])));
370         } else {
371             A = (pi * a ** 3 - po * b ** 3) / (b ** 3 - a ** 3); B = ((pi - po) * a ** 3 * b ** 3) / (2 * (b ** 3 - a ** 3));
372             sr = r.map((q) => A - (2 * B) / q ** 3); st = r.map((q) => A + B / q ** 3); sz = st;
373             u = r.map((q, i) => (q / E) * (st[i] - nu * (st[i] + sr[i])));
```

sims.js, lines 366-373

Note. The thin-wall error against Lamé is shown: about 0.1% for $t/a = 0.06$, a few tens of percent for a thick wall. Equal inside and outside pressure gives $\sigma_\theta = \sigma_r = -p$ and zero von Mises stress, as expected for hydrostatic compression.

6.6 Thermal stress (Chapter 6.6)

Physics. A temperature change produces a free strain $\alpha\Delta T$ and no stress unless expansion is prevented. Fully preventing it in one direction gives $\sigma = -E\alpha\Delta T$. In a thin plate prevented in two directions (biaxial) the lateral strains are zero: $\epsilon_x = [\sigma - \nu\sigma]/E + \alpha\Delta T = 0$, so $\sigma = -E\alpha\Delta T/(1-\nu)$. In a block prevented in all three directions, $\sigma = -E\alpha\Delta T/(1-2\nu)$. A partial restraint f (between 0 and 100%) scales the stress, and the free expansion is $\alpha\Delta T\, L(1-f)$.

```
412         const factor = { '1d': 1, '2d': 1 / (1 - nu), '3d': 1 / (1 - 2 * nu) };
413         const sth = (c, dT) => -E * al * dT * f * factor[c];
414         const sT = sth(v.case, v.dT), sTot = sT + v.smech;
```

sims.js, lines 412-414

6.7 Failure theories (Chapter 11)

Physics. Four theories predict yielding or fracture from a state of stress by comparing a single measure with the uniaxial strength σ_{cr}. *Rankine* (maximum principal stress): $\max|\sigma_i| \le \sigma_{cr}$. *St Venant* (maximum normal strain): $\epsilon_1 = [\sigma_1 - \nu(\sigma_2+\sigma_3)]/E \le \sigma_{cr}/E$. *Tresca* (maximum shear): $\sigma_1 - \sigma_3 \le \sigma_{cr}$. *von Mises* (distortional energy): the equivalent stress of Chapter 2 is at most σ_{cr}. For a ductile metal, Tresca and von Mises predict yield and agree for uniaxial tension (both σ) and differ most in pure shear, where Tresca gives $\tau = \sigma_y/2$ and von Mises $\tau = \sigma_y/\sqrt{3}$.

```
449         const [a, b, c] = [v.s1, v.s2, v.s3].sort((p, q) => q - p), { scr: S, nu } = v;
450         const rank = Math.max(...[a, b, c].map(Math.abs));
451         const tre = a - c;
452         const vm = Math.sqrt(0.5 * ((a - b) ** 2 + (b - c) ** 2 + (c - a) ** 2));
453         const sv = Math.max(...[[a, b, c], [b, a, c], [c, a, b]].map(([i, j, k]) => Math.abs(i - nu * (j + k))));
454         const SFs = [['Max principal stress', S / rank], ['Max normal strain', S / sv], ['Max shear (Tresca)', S / tre],
            ↪  ['Distortional energy (von Mises)', S / vm]].map(([n, q]) => [n, Number.isFinite(q) ? q : Infinity]);
455         const minSF = Math.min(...SFs.map((q) => q[1])), gov = SFs.find((q) => q[1] === minSF)[0];
456         const t = linspace(0, 2 * Math.PI, 181);
```

sims.js, lines 449-456

Code. The safety factor of each theory is σ_{cr} divided by its equivalent stress. The governing theory is the one with the smallest value. The input principal stresses are sorted first, so the order in which they are typed does not matter. In the (σ_1, σ_2) plane at $\sigma_3 = 0$ the envelopes are drawn: a square (Rankine), a skewed square (St Venant), a hexagon (Tresca) and an ellipse (von Mises) that circumscribes it; the state is plotted as a cross.

6.8 Energy methods (Chapter 10)

Physics. For a linear elastic body the work done by the loads is stored as strain energy, $U = \frac{1}{2}\sum P_i\delta_i$, and depends only on the final loads. Castigliano's first theorem follows: $\delta_i = \partial U/\partial P_i$. Examples: axial bar $U = P^2L/2AE$; shaft $U = T^2L/2GJ$; bending $U = \int M^2/2EI\,dx$; shear $U = \int V^2/(2k_sGA)\,dx$. Linearity implies Maxwell–Betti reciprocity: the deflection at b due to a unit load at a equals the deflection at a due to a unit load at b.

Code. The simulator evaluates $U(P)$ in closed form, differentiates it *numerically* by a central difference, and compares with the closed-form displacement, so the theorem is verified rather than assumed. The reciprocity check uses the influence function of a simply supported beam at $0.3L$ and $0.7L$.

```
492         const U = (P) => ({
493             axial: (P * P * L) / (2 * A * E), torsion: (P * 1000) ** 2 * L / (2 * G * J),
494             cant: (P * P * L ** 3) / (6 * E * I) + (sh ? (P * P * L) / (2 * ks * G * A) : 0),
495             ss: (P * P * L ** 3) / (96 * E * I) + (sh ? (P * P * L) / (8 * ks * G * A) : 0),
496         }[v.case]);
```

sims.js, lines 492-496

Note. For the shaft the load is a torque in N m and the displacement is the twist; both Castigliano and closed-form values are shown as $\phi \times 10^3$ (millirad). Earlier these two numbers differed by 10^6 because of a unit slip in the closed form; it was found by the limiting-case tests and corrected.

6.9 Column buckling (Chapter 9.4)

Physics. A straight column under axial load becomes unstable when the load reaches Euler's value; the bent shape then becomes an equilibrium state. With effective length KL ($K = 1,\ 0.699,\ 0.5,\ 2$ for pinned–pinned, fixed–pinned, fixed–fixed and fixed–free),

$$P_{cr} = \frac{\pi^2 EI}{(KL)^2}, \qquad \sigma_{cr} = \frac{\pi^2 E}{\lambda^2}, \qquad \lambda = \frac{KL}{r},\ r = \sqrt{I/A}, \tag{6.5}$$

with I the *smaller* second moment of area (buckling occurs about the weak axis). Euler's formula assumes that the column stays elastic, so it only holds for $\lambda > \lambda_c = \pi\sqrt{E/\sigma_y}$. A short column fails by yielding with some bending, which the Johnson parabola captures:

$$\sigma_{cr} = \sigma_y\Big[1 - \frac{\sigma_y \lambda^2}{4\pi^2 E}\Big], \qquad \lambda < \lambda_c, \tag{6.6}$$

which meets Euler's curve at λ_c where both equal $\sigma_y/2$.

```
543         const m = mat(v), E = m.E * 1000, K = { pp: 1, fp: 0.699, ff: 0.5, fr: 2 }[v.end], L = v.L * 1000;
544         const Imin = (Math.min(v.b, v.h) ** 3 * Math.max(v.b, v.h)) / 12, A = v.b * v.h, r = Math.sqrt(Imin / A);
545         const Pcr = (Math.PI ** 2 * E * Imin) / (K * L) ** 2, lam = (K * L) / r, scr = Pcr / A;
546         const lamC = Math.PI * Math.sqrt(E / m.sy), valid = lam >= lamC;
547         const sJ = m.sy * (1 - (m.sy * lam * lam) / (4 * Math.PI ** 2 * E)); // Johnson parabola, joins Euler at lambdac with sigma =
        ↪  sigmay/2
548         const sdes = valid ? scr : Math.max(sJ, 0), Pdes = sdes * A;      // Euler stress exceeds yield for short columns, so it
        ↪  cannot be used there
549         const sf = v.P > 0 ? Pdes / (v.P * 1000) : Infinity;
550         const lams = linspace(10, 250, 120);
551         const x = linspace(0, 1, 101);
```

sims.js, lines 543-551

Note. The Johnson branch was added after limiting-case testing showed that, for a stocky column, Euler's formula gave a critical stress above 4×10^4 MPa and a safety factor based on it. The "Euler load" readout is still Euler's value; the "design critical load" is the governing one.

7 The finite element method

7.1 Why a finite element tab

The half-space solution is exact only for an infinite body; the simulators are exact only for simple members. The finite element method (FEM) solves the same governing equations numerically on a body of any shape. It is not part of the syllabus, but it is built entirely from the syllabus' equations, and it provides an independent check of the half-space model (Section 7.5). A separate primer, `docs/fem_primer.pdf`, explains the method from the beginning; the summary here is what is needed to read the code.

7.2 The method in seven steps

1. **Mesh**: cut the body into elements joined at nodes.
2. **Unknowns**: the displacement components $\mathbf{u}$ at the nodes (the degrees of freedom).
3. **Shape functions**: inside an element, $\mathbf{u}(\mathbf{x}) = \sum_i N_i(\mathbf{x})\mathbf{u}_i$ with $N_i = 1$ at node i and 0 at the others.
4. **Strain and stress**: $\boldsymbol{\epsilon} = \mathbf{B}\mathbf{u}_e$ (Chapter 5, $\mathbf{B}$ contains derivatives of N_i) and $\boldsymbol{\sigma} = \mathbf{D}\boldsymbol{\epsilon}$ (Chapter 6, $\mathbf{D}$ is Hooke's law in Voigt form).
5. **Stiffness**: from the principle of virtual work or minimum potential energy (Chapter 10), $\mathbf{k}_e = \int_{V_e} \mathbf{B}^T\mathbf{D}\mathbf{B}\, dV$; the integral is evaluated numerically at Gauss points.
6. **Assembly and solution**: add all $\mathbf{k}_e$ into one global matrix $\mathbf{K}$ (symmetric by Maxwell–Betti) and solve $\mathbf{K}\mathbf{u} = \mathbf{f}$ after removing the supported degrees of freedom; otherwise $\mathbf{K}$ is singular because the body could move freely.
7. **Recovery**: compute $\boldsymbol{\epsilon}$ and $\boldsymbol{\sigma}$ from $\mathbf{u}$, then average or extrapolate to nodes.

Code. The common linear algebra is the same for every case. Eliminating the constrained degrees of freedom, solving the free system, and returning the reaction forces $\mathbf{R} = \mathbf{K}\mathbf{u} - \mathbf{f}$ (which must balance the applied load) takes seven lines.

```
40 def solve_system(K, f, fixed):
41     """Partition out the constrained DOFs (supports) and solve K_ff u_f = f_f; also return reactions."""
42     n = K.shape[0]
43     free = np.setdiff1d(np.arange(n), fixed)
44     u = np.zeros(n)
45     u[free] = np.linalg.solve(K[np.ix_(free, free)], f[free])
46     return u, K @ u - f
```

fem.py, lines 40-46

7.3 One-dimensional elements

Tapered bar. A two-node element has a linear displacement, so its strain is constant and its stiffness is $EA/h\,[[1,-1],[-1,1]]$. For a bar whose area varies linearly, the exact displacement is $u(x) = \frac{P}{Em}\ln\frac{A(x)}{A_0}$; the FEM, using the area at the element midpoint, converges as h^2 (the ratio to exact is $0.935, 0.980, 0.995, 0.9987$ for 1, 2, 4, 8 elements).

```
77     K = np.zeros((n + 1, n + 1))
78     for e in range(n):
79         ke = E * A(0.5 * (xs[e] + xs[e + 1])) / hx * np.array([[1, -1], [-1, 1]])
80         K[e:e + 2, e:e + 2] += ke
81     f = np.zeros(n + 1); f[-1] = P
82     fixed = np.array([0])
```

fem.py, lines 77-82

Beam. The Euler–Bernoulli element has two unknowns at each node, deflection v and slope θ, and a cubic displacement within the element (Hermite functions). Its stiffness matrix follows from $\int EI\, N''^T N''\, dx$. Because the cubic is the exact solution of $EIv'''' = 0$ between loads, the deflection at the nodes is *exact* for

any number of elements (for a point load; for a distributed load the nodal values are exact when consistent nodal loads are used, which the code does), while the bending moment, which is EIv'' and so only linear inside an element, improves with refinement.

```
def hermite(xi, h, v1, t1, v2, t2):
    N1 = 1 - 3 * xi**2 + 2 * xi**3; N2 = xi - 2 * xi**2 + xi**3
    N3 = 3 * xi**2 - 2 * xi**3;     N4 = -xi**2 + xi**3
    v = N1 * v1 + N2 * h * t1 + N3 * v2 + N4 * h * t2
    c = ((-6 + 12 * xi) * v1 + h * (-4 + 6 * xi) * t1 + (6 - 12 * xi) * v2 + h * (-2 + 6 * xi) * t2) / h**2
    return v, c
```

fem.py, lines 141-146

```
    ke = EI / hx**3 * np.array([[12, 6 * hx, -12, 6 * hx], [6 * hx, 4 * hx**2, -6 * hx, 2 * hx**2],
                                [-12, -6 * hx, 12, -6 * hx], [6 * hx, 2 * hx**2, -6 * hx, 4 * hx**2]])
    for e in range(n):
```

fem.py, lines 157-159

7.4 Two-dimensional elements

Physics. In plane stress ($\sigma_z = \tau_{xz} = \tau_{yz} = 0$, a thin plate) the constitutive matrix in Voigt form is

$$\mathbf{D} = \frac{E}{1-\nu^2}\begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix}. \tag{7.1}$$

The **constant-strain triangle (CST)** has linear displacement, so $\mathbf{B}$ is constant: $\mathbf{B} = \frac{1}{2A}[\dots]$ and $\mathbf{k}_e = tA\,\mathbf{B}^T\mathbf{D}\mathbf{B}$. It is simple but too stiff in bending. The **four-node quadrilateral (Q4)** has bilinear displacement and an element mapped to the unit square by the isoparametric map; $\mathbf{B}$ varies and the stiffness is integrated by 2×2 Gauss points.

```
def q4_B(xy, xi, eta):
    dN = 0.25 * np.array([[-(1 - eta), (1 - eta), (1 + eta), -(1 + eta)],
                          [-(1 - xi), -(1 + xi), (1 + xi), (1 - xi)]])
    J = dN @ xy
    dNdx = np.linalg.solve(J, dN)
    B = np.zeros((3, 8))
    B[0, 0::2] = dNdx[0]; B[1, 1::2] = dNdx[1]
    B[2, 0::2] = dNdx[1]; B[2, 1::2] = dNdx[0]
    return B, np.linalg.det(J)
```

fem.py, lines 253-261

Code. `fe2d` loops over the elements, accumulates $\mathbf{k}_e$ into $\mathbf{K}$ with `np.ix_`, solves, and recovers stress. For Q4 the stress is evaluated at the four Gauss points and *extrapolated* to the element corners by the bilinear matrix `_EXTRAP`; at each node the contributions of all elements sharing it are averaged (equal weights for Q4, area weights for CST).

```
# extrapolation matrix: Gauss-point stresses -> element corner stresses (bilinear, Gauss points at +-1/sqrt3)
_SGN = np.array([[-1, -1], [1, -1], [1, 1], [-1, 1]], float)
_EXTRAP = np.array([[0.25 * (1 + gs[0] * c[0] * ROOT3) * (1 + gs[1] * c[1] * ROOT3) for gs in _SGN] for c in _SGN])
```

fem.py, lines 273-275

Plane-stress cantilever. A beam of length L and depth h fixed at one end and loaded by an end shear. The reference solutions are Euler–Bernoulli ($\delta = PL^3/3EI$) and Timoshenko ($+PL/k_sGA$). With a clamped end the 2D solution contains stress concentrations at the root corners that beam theory does not, so the comparison is made with σ_x at mid-length, away from the clamp.

Plate with a hole. One quarter of a plate in uniaxial tension is meshed with a graded polar-style grid: radial layers from the hole of radius a to the plate edge of half-width W, with the outer edge following the square. The symmetry planes become supports ($u_x = 0$ on $x = 0$, $u_y = 0$ on $y = 0$). Kirsch's solution for an infinite plate gives $\sigma_\theta = \sigma_\infty\left[1 + \frac{a^2}{2r^2} + \frac{3a^4}{2r^4}\right]$ on the line through the hole perpendicular to the load, so $K_t = 3$ at the edge; a finite plate has a slightly larger K_t (3.10 for $W/a = 12$, 3.60 for $W/a = 4$ with Q4).

```
420 def hole_mesh(a, W, n_theta, n_r, grade=1.12):
421     th = np.linspace(0, np.pi / 2, n_theta + 1)
422     s = (grade ** np.arange(n_r + 1) - 1) / (grade**n_r - 1)
423     rout = W / np.maximum(np.cos(th), np.sin(th))
424     nodes = np.array([[(a + (rout[j] - a) * s[i]) * np.cos(th[j]), (a + (rout[j] - a) * s[i]) * np.sin(th[j])]
425                       for i in range(n_r + 1) for j in range(n_theta + 1)])
426     idx = lambda i, j: i * (n_theta + 1) + j
427     quads = [[idx(i, j), idx(i + 1, j), idx(i + 1, j + 1), idx(i, j + 1)] for i in range(n_r) for j in range(n_theta)]
428     return nodes, quads, idx, th
```

fem.py, lines 420-428

7.5 The three-dimensional block

Physics. A cuboid with a square patch pressure q on its top face is solved as a three-dimensional elasticity problem. Only a quarter is meshed, using the two symmetry planes ($u_x = 0$ on $x = 0$, $u_y = 0$ on $y = 0$) as supports; the base is either a smooth rigid surface ($u_z = 0$) or fixed. The side faces and the unloaded part of the top are *free*: the finite element weak form leaves the traction on them equal to zero without being asked (a *natural* boundary condition). This is precisely what the half-space solution cannot do.

Code. Elements are uniform 8-node bricks. The Gauss rule is $2 \times 2 \times 2$. Because all bricks are identical, `brick` builds one 24×24 stiffness matrix and one set of **B** matrices that are reused everywhere. The **D** matrix is the full three-dimensional Hooke matrix in the order xx, yy, zz, yz, xz, xy.

```
37 def brick(hx, hz, D):
38     """Stiffness matrix of a uniform brick, B matrices at the Gauss points, and the Gauss-to-corner extrapolation."""
39     jac = np.array([hx / 2, hx / 2, hz / 2])
40     ke = np.zeros((24, 24)); Bg = []
41     for g in _C:
42         p = g / ROOT3
43         dNdx = _dn(*p) / jac[:, None]
44         B = np.zeros((6, 24))
45         B[0, 0::3] = dNdx[0]; B[1, 1::3] = dNdx[1]; B[2, 2::3] = dNdx[2]
46         B[3, 1::3] = dNdx[2]; B[3, 2::3] = dNdx[1]
47         B[4, 0::3] = dNdx[2]; B[4, 2::3] = dNdx[0]
48         B[5, 0::3] = dNdx[1]; B[5, 1::3] = dNdx[0]
49         Bg.append(B); ke += B.T @ D @ B * np.prod(jac)
50     M = np.array([_n(*(c * ROOT3)) for c in _C])      # corner c from Gauss point g: N_g evaluated at sqrt(3) * corner
51     return ke, np.array(Bg), M
```

fem3d.py, lines 37-51

Code. A 3D mesh with thousands of nodes would be too large for a dense solver, and `scipy` is not installed, so the system is solved *matrix-free* by conjugate gradients with a Jacobi (diagonal) preconditioner. The matrix **K** is never formed: the product **Kx** is assembled brick by brick, by gathering the nodal values of **x**, multiplying by the 24×24 matrix, and scattering back with `np.bincount`.

```
54 def cg(matvec, b, dinv, free, tol=1e-9, maxit=6000):
55     x = np.zeros_like(b); r = np.where(free, b, 0.0); z = dinv * r; p = z.copy(); rz = r @ z; b0 = np.linalg.norm(r) or 1.0
56     for it in range(1, maxit + 1):
57         Ap = np.where(free, matvec(p), 0.0)
58         alpha = rz / (p @ Ap)
59         x += alpha * p; r -= alpha * Ap
60         if np.linalg.norm(r) < tol * b0:
61             return x, it, float(np.linalg.norm(r) / b0)
62         z = dinv * r; rn = r @ z; p = z + rn / rz * p; rz = rn
63     return x, maxit, float(np.linalg.norm(r) / b0)
64
65
66 # ------------------------------------------------------------------ half-space reference (tension positive)
```

fem3d.py, lines 54-66

```
131     matvec = lambda x: np.bincount(dofs.ravel(), ((x[dofs]) @ ke).ravel(), minlength=ndof)
132     diag = np.bincount(dofs.ravel(), np.tile(np.diag(ke), len(conn)), minlength=ndof)
133     dinv = np.where(free, 1 / diag, 0.0)
134     u, iters, res = cg(matvec, f, dinv, free)
```

fem3d.py, lines 131-134

Physics. Reference solutions. The half-space reference is exact for a load on an infinite body. For the vertical stress under the centre of a square patch of half-width a the Newmark result is $\sigma_z = 4q\,I(m,n)$ with $m = n = a/z$ and

$$I = \frac{1}{4\pi}\left[\frac{2mn\sqrt{m^2+n^2+1}}{m^2+n^2+m^2n^2+1}\,\frac{m^2+n^2+2}{m^2+n^2+1} + \tan^{-1}\frac{2mn\sqrt{m^2+n^2+1}}{m^2+n^2+1-m^2n^2}\right] \tag{7.2}$$

(corner factor of a rectangle; the factor 4 sums the four quarters). The side-face stress and the horizontal profile use a numerical superposition of Boussinesq point loads over the patch.

```
67 def newmark_centre(a, z, q):
68     """sigma_z (compression) under the centre of a uniform square load of half-width a, depth z (Newmark corner factor x 4)."""
69     m = n = a / z
70     s = m * m + n * n + 1
71     I = (2 * m * n * np.sqrt(s) / (s + m * m * n * n) * (s + 1) / s + np.arctan2(2 * m * n * np.sqrt(s), s - m * m * n * n)) / (4 *
       ↪  np.pi)
72     return 4 * q * I
```

fem3d.py, lines 67-72

Note. The brick element is stiff in bending and for nearly incompressible materials (volumetric locking), and stresses at the loaded surface are least accurate because they are extrapolated from Gauss points inside the brick (about 14% overshoot at the loaded face with the default mesh). The block is therefore used for the *trends* it shows: side-face stress about 0.4% of q with FEM against about 2.6% with the half-space, and a centre stress that approaches the half-space value as the block grows (1.26, 1.03 for a 2×3 m and an 8×8 m block).

7.6 The FEM user interface

The tab is built by `fem.js`. It keeps a table of the five problems with their sliders, sends the inputs to `POST /api/fem/solve`, and receives a common response: a mesh (nodes, deformed nodes, polygons, values for each stress field), chart data, a comparison table (FEM versus reference), diagnostics and, for the bar and beam, the stiffness matrix. The mesh is drawn on an HTML canvas coloured by the stress ramp; the charts use Plotly; and the panel "What the solver did" states, with the actual numbers of the current solve, the seven steps above. A request counter ignores late responses, as in the section plots.

```
async function solve() {
    const my = ++store.seq;
    if (store.ctrl) store.ctrl.abort();
    store.ctrl = new AbortController();
    const [, E, nu] = MATERIALS[store.material];
    const body = { case: store.c.id, E_GPa: E, nu, ...store.values };
    try {
        const res = await fetch('/api/fem/solve', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body:
        ↪  JSON.stringify(body), signal: store.ctrl.signal });
        if (!res.ok) throw new Error('HTTP ' + res.status);
        const data = await res.json();
        if (data.error) throw new Error(data.error);
        if (my !== store.seq) return;
        store.data = data; render();
    } catch (e) {
        if (e.name === 'AbortError' || my !== store.seq) return;
        console.error('FEM request failed:', e);
        const st = $('fem-status'); st.className = 'sim-status bad';
        st.innerHTML = '<span class="badge">No result</span><span class="summary">The FEM server did not respond. Start it with
        ↪  ./run_shape_analytics.sh.</span>';
    }
}

// ------------------------------------------------------------------ rendering
```

fem.js, lines 192–213

8 Verification

8.1 Approach

A simulator can be wrong in ways that look reasonable: a plausible picture with a wrong number. The checks here therefore compare the code with something independent: a closed-form result, a limiting case where the answer is known, an invariant that must hold for *any* input, or a second implementation of the same physics. Where an exact answer is not available, a numerical experiment (mesh refinement, growing the body) shows the trend toward one.

Component	Checks	What was verified
`physics.js`	54	invariants and ordering of the principal stresses for 4000 random tensors over 12 decades of stress; hydrostatic, uniaxial, pure-shear and double-root cases; Boussinesq and Cerruti satisfy $\nabla\cdot\boldsymbol{\sigma}=\mathbf{0}$ at random points for several ν; the Boussinesq stress equals Hooke's law applied to the gradient of its displacement; $\int\sigma_z\,dA=-P$ on four depths; the surface is stress-free outside the load; far-field decay; singular points stay finite
`server.py`	40	eigen-decomposition against numpy; $\lvert\mathbf{t}\rvert^2=\sigma_{nn}^2+\tau^2$; the section is linear in load, obeys the triangle inequality for superposed loads, is symmetric, is zero for zero load, is hydrostatic for full coverage, has the right cylinder and sphere masks; and the browser physics agrees with the server for four load orientations to 2×10^{-4}
Simulators	110	each closed form (Chapters 2 and 6) plus limits, listed below
Simulators, random	22 500	2500 random or extreme inputs for each of nine simulators: no exception, no NaN, and the only infinite values are safety factors with no load
FEM	43	patch test, closed forms, equilibrium, full-pressure block, growing the block toward the half-space, 116 extreme inputs
3D tab	94	every shape, material, load type and map with painted loads, no non-finite output, no console error

8.2 Limiting cases of the simulators

Simulator	Limits checked
Mohr	uniaxial ($\sigma_1 = \sigma,\ \tau_{\max} = \sigma/2$); hydrostatic ($\tau_{\max} = 0,\ \sigma_{vM} = 0$); pure shear ($\sigma_{vM} = \sqrt{3}\tau$); 400 random states: the transformation law, $\sigma_{x'} + \sigma_{y'}$ invariant, $\sigma_1 + \sigma_2 + \sigma_3 = I_1$, $\sigma_1\sigma_2\sigma_3 = I_3$, $\sigma_{vM}^2 = I_1^2 - 3I_2$
Beam	simply supported central load ($M = PL/4,\ \delta = PL^3/48EI$), UDL ($wL^2/8,\ 5wL^4/384EI$), cantilever tip load, UDL and end moment, moment at a support, equilibrium of reactions, load at a support, $a > L$ clamp, zero load; Timoshenko shear term $= PL/4k_sGA$, tends to Euler–Bernoulli for slender beams and dominates for stubby ones; $1.5V/A$, $4V/3A$ and $2V/A$ (thin tube) shear stresses; circle and tube properties
Shaft	$16T/\pi D^3$, TL/GJ, hollow J, $\tau_{\min} = \tau_{\max} d/D$, thin-wall $T/2\pi r^2 t$, zero torque, solid limit, linearity, $\phi \propto L/D^4$
Axial	elongation $= \sum NL/EA$, fixed–fixed net elongation zero, thermal $-E\alpha\Delta T$ (fixed) and $\alpha\Delta T\, L$ (free)
Pressure vessel	Lamé cylinder and sphere inner and outer hoop stress, external pressure, equal pressures (hydrostatic), zero pressure, thin-wall limit, inflation $u = \frac{a}{E}[\sigma_\theta - \nu(\sigma_r + \sigma_z)]$, von Mises from the Lamé stresses
Thermal	$-E\alpha\Delta T$, $/(1-\nu)$, $/(1-2\nu)$; half restraint gives half the stress; $\Delta T = 0$; cooling gives tension; ΔT to yield
Failure	uniaxial tension and compression: all four theories agree; pure shear: Tresca $\sigma_y/2\tau$, von Mises $\sigma_y/\sqrt{3}\tau$, Rankine σ_y/τ, St Venant $\sigma_y/[\tau(1+\nu)]$; hydrostatic: Tresca and von Mises never fail; equal biaxial tension; unsorted input; doubling the stresses halves each factor; Tresca $\geq$ von Mises $\geq \frac{\sqrt{3}}{2}$Tresca
Energy	all four members with and without shear: Castigliano equals closed form, $U = \frac{1}{2}P\delta$, reciprocity $\delta_{ab} = \delta_{ba}$
Buckling	$\pi^2 EI/(KL)^2$ for the four end conditions, weak-axis choice, $1/L^2$ scaling, transition slenderness $\pi\sqrt{E/\sigma_y}$, short columns flagged and capped at yield, zero load, slender column

8.3 Verification of the finite element code

Test	Result
Patch test, CST and Q4, irregular mesh	uniform stress reproduced to 10^{-8}
Uniform bar	exact at any mesh
Tapered bar	ratio to exact $0.935 \to 0.9999$ (1 to 32 elements)
Beam, tip load and UDL	nodal deflection exact to 10^{-9} with 1 element, $L/h = 2$ and 250
Plane-stress cantilever, Q4, $\nu = 0 \ldots 0.499$	within 1.7% of Timoshenko on a 12-element depth mesh
Coarse slender cantilever ($L/h = 40$)	Q4 0.89 and CST 0.54 of theory: shear locking is real, and noted
Plate with a hole	$K_t = 3.10$ ($W/a = 12$), 3.17 (8), 3.60 (4): the finite-width trend toward 3
Block, full-face pressure	$\sigma_z = -q$ everywhere, $\sigma_x = \sigma_y = 0$, crush $-qH/E$
Block, patch load	centre stress 1.26, 1.06, 1.03 times the half-space value for blocks of 2×3, 4×4, 8×8 m; side-face σ_x about 0.4% of q against 2.6%
Reactions	equal the applied load to rounding in all cases and on 116 extreme inputs

8.4 Defects found by the tests, and what was changed

Each of the following was found by a limiting-case test and then corrected; the original behaviour is stated so that the reason for the code can be understood.

1. **Torsion in the energy tab** (`sims.js`, `energy.compute`). The closed-form displacement and the work were 10^6 too small because the torque was converted twice. The status check had a special case that skipped torsion, which hid the error. Both corrected.

2. **Short columns** (`buckling.compute`). Euler's critical stress was reported even above yield (up to 4×10^4 MPa) and the safety factor used it. A Johnson parabola now governs when $\lambda < \lambda_c$.

3. **Principal stresses** (`physics.js`, `principalStresses`; `sims.js`, `eig3`). A fixed threshold on J_2 treated small non-hydrostatic stress states as hydrostatic. The threshold is now relative to the stress scale.

4. $\nu = 0.5$ (`tangentialTensor` and `server.py`). The Lamé constant divides by $1 - 2\nu$: infinite stress in the browser and a division-by-zero error on the server. The value is capped at 0.499 in that formula.

5. **Section endpoint**. A resolution below 2 crashed the request and a force direction that was not a unit vector scaled the load. Resolution is clamped (2 to 400) and the direction is normalised. Earlier in the project the section used the wrong axis for cylinders and drew stress outside the body; both were corrected.

6. **Display**. An infinite safety factor appeared as a dash and now shows ∞. Strains above 1% carry a warning.

7. **Finite plate with a hole** (not a defect but a result): the first Kirsch comparison showed K_t above 3; the trend with the width ratio explains this.

9 Limitations and how to extend the project

9.1 Limits of validity

- **Linear elastic and small strain** everywhere. Plasticity, large rotation, contact, dynamics and damage are not modelled. Rubber in the 3D tab can exceed 100% strain; the numbers are then indicative only.
- **Half-space** (3D tab, section): an infinite body. For a finite one it leaves stress on the sides and base. The FEM block quantifies this and shows the model is good for a deep, wide body. A *partial* patch is not self-equilibrated in the 3D tab (there are no supports), and displacements are measured from the bottom centre.
- **Beams and columns**: numerical integration on 401 points; constant shear factor; ideal straight columns. The Mohr tool accepts symmetric stress matrices only.
- **FEM**: plane stress only (no plane strain or axisymmetric element); CST and Q4 are stiff in bending until refined; bricks lock for nearly incompressible materials; the dense 2D solver is capped near 3600 degrees of freedom and the 3D block near 9000 nodes; surface stress values are extrapolated; the block is a cuboid with a square patch aligned to the mesh.
- **Environment**: Three.js, Plotly and the fonts come from CDNs, so the interface needs network access.

9.2 Adding a simulator

1. In `sims.js` define an object with the fields shown in Chapter 6; the physics goes in `compute(v)`, returning readouts, charts and equations in the stated format.
2. Add it to the `SIMS` array; the navigation, controls, A/B comparison, exports and routing follow automatically.
3. Check it against at least two closed forms and the limits of every input (zero, minimum, maximum), since the safety factors and ratios divide by quantities that can be zero.

9.3 Removing the FEM tab

It is self-contained: delete `fem.js`, `fem.css`, `fem.py` and `fem3d.py`; remove the `fem.css` link and the `fem.js` script tag from `index.html`; remove the two `fem_router` lines from `server.py`; and remove the `registerModule` function in `sims.js`. Nothing else depends on it. `docs/FEM_PLAN.md` describes the design and the staged scope.

9.4 Ideas for further work

Plane-strain and axisymmetric elements (to compare with the Lamé vessel); thermal loads in the FEM; a finite-element field for cylinders and spheres (needs a mesher); a unit switch; and wiring the FEM block result into the 3D tab as an alternative to the half-space solution.

A Index: physics principle to code

Principle (course chapter)	File	Function or object
Cauchy's formula, normal and shear traction (2, 4)	`server.py`	`compute_traction`
Principal stresses, invariants (4)	`physics.js`; `server.py`; `sims.js`	`principalStresses`; `compute_principal` (`eigh`); `eig3`
Von Mises, Tresca (11)	`physics.js`; `sims.js`	`vonMises`; `mohr`, `failure .compute`
Strain from stress, Hooke (5, 6)	`physics.js`	`strainNorm`; `tangentialTensor`
Boussinesq normal load (3, 6)	`physics.js`	`boussinesqTensor`, `halfSpaceDisplacement`
Cerruti tangential load (5, 6)	`physics.js`	`halfSpaceDisplacement`, `tangentialTensor`
Superposition (10.1)	`app.js`; `server.py`	`computeStressField` (loop over `infos`); `compute_section` (loop over `req.loads`)
Tensor rotation (2.5)	`app.js`; `server.py`	`rot`; `g(p, q)`
Hydrostatic compression (2, 6)	`app.js`; `server.py`	the `full` test (coverage ≥ 0.99)
Statics and area (1–2)	`app.js`	`computeVertexAreas`, `paintedAreaOf`
Beam statics, $EIv'' = M$, Timoshenko (8, 9)	`sims.js`	`beam.compute`, `section`, `cumtrapz`
Torsion (7)	`sims.js`	`torsion.compute`, `draw3d`
Statically indeterminate bar (6, 7)	`sims.js`	`axial.compute`
Lamé cylinder and sphere (7)	`sims.js`	`pressure.compute`
Thermal stress (6.6)	`sims.js`	`thermal.compute`
Failure theories (11)	`sims.js`	`failure.compute`
Castigliano, reciprocity (10)	`sims.js`	`energy.compute`
Euler and Johnson buckling (9.4)	`sims.js`	`buckling.compute`
Bar and beam elements (energy methods)	`fem.py`	`run_bar`, `run_beam`, `hermite`
Plane stress, **B**, **D** (5, 6)	`fem.py`	`d_matrix`, `q4_B`, `cst_B`, `fe2d`
3D elasticity, bricks (5, 6)	`fem3d.py`	`d_matrix3`, `brick`, `run_block`
Equilibrium of reactions (3)	`fem.py`, `fem3d.py`	`solve_system`, `diagnostics`
Half-space reference (3, 6)	`fem3d.py`	`newmark_centre`, `halfspace`

B Interfaces and data formats

B.1 HTTP API

Route	Purpose
`POST /api/traction`	input: six stress components and a normal; output: traction vector, normal stress, shear vector and magnitude
`POST /api/principal`	input: six stress components; output: principal stresses and directions, $\tau_{\max}$, von Mises stress
`POST /api/section`	input: loads (force, painted points, load point and normal, direction, area, coverage), slice plane and position, body extents and shape, resolution, ν, E; output: grid axes and four stress fields (null outside the body)
`POST /api/fem/solve`	input: case (`bar`, `beam`, `cantilever`, `hole`, `block`), element, mesh density and the case parameters; output: mesh view, charts, comparison table, diagnostics, optional stiffness matrix
`GET /` and static files	the single page application

B.2 Symbols

$\boldsymbol{\sigma}, \boldsymbol{\epsilon}$	stress, strain tensors	E, ν	Young's modulus, Poisson's ratio
$\sigma_1 \geq \sigma_2 \geq \sigma_3$	principal stresses	G, K, λ	shear, bulk modulus, Lamé constant
I_1, I_2, I_3	stress invariants	$\mathbf{B}, \mathbf{D}$	strain–displacement and constitutive matrices
J_2, J_3	deviatoric invariants	$\mathbf{K}, \mathbf{f}, \mathbf{u}$	global stiffness, load and displacement vectors
$\tau_{\max}$	maximum shear stress	k_s	shear correction factor
σ_{vM}	von Mises stress	λ	slenderness KL/r (buckling)
P, q, p	force, patch pressure, pressure	K_t	stress concentration factor